Your Institute Name

Department of Information Technology

B.Tech Project Report

SimpleTM++: Adaptive Hybrid Tokenization for Multivariate Time Series Forecasting

Submitted by: Your Name
Roll Number: Your Roll Number
Degree: Bachelor of Technology
Academic Year: 2025–2026

Under the guidance of: Dr./Prof. Supervisor Name

March 9, 2026

Declaration

I hereby declare that the work presented in this report titled **"SimpleTM++: Adaptive Hybrid Tokenization for Multivariate Time Series Forecasting"** is being submitted as part of my B.Tech Project work under the guidance of **Dr./Prof. Supervisor Name**. The work described in this report is based on my study, implementation, experimentation, and analysis carried out during the academic session 2025–2026.

Student Name: Your Name
Roll Number: Your Roll Number

Signature: _____

Acknowledgement

I would like to express my sincere gratitude to my project guide, **Dr./Prof. Supervisor Name**, for constant support, guidance, and valuable suggestions throughout the course of this work. I also thank the faculty members of the **Department of Information Technology** for providing an academic environment suitable for conducting this project.

I am grateful to my peers and all those who directly or indirectly contributed to the progress of this work.

Abstract

Multivariate Time Series Forecasting (MTSF) is a fundamental problem with applications in energy demand prediction, weather forecasting, traffic analysis, industrial monitoring, and finance. Recent forecasting approaches based on large Transformer architectures have shown strong performance, but such methods often require significant computational resources. In contrast, the SimpleTM framework proposes a lightweight architecture using wavelet-based tokenization and geometric attention for multivariate forecasting.

This project studies whether the tokenization stage in SimpleTM can be improved or generalized. The central hypothesis is that different time-series datasets exhibit different dominant structures such as periodicity, local fluctuations, and non-stationarity, and therefore a single fixed tokenization scheme may not be optimal across all datasets. Motivated by this observation, the project investigates an FFT-based tokenizer as an alternative to the original wavelet-based tokenizer and further aims to develop a hybrid tokenization framework that combines wavelet, FFT, and local convolutional representations.

The work completed so far includes literature review, understanding of the SimpleTM codebase, exploratory data analysis on benchmark datasets, implementation of an FFT-only tokenizer baseline, debugging of repository and training issues, and initial experiments on the ETTh1 dataset. The FFT-only model trains successfully and provides a valid baseline, although it currently underperforms the original SimpleTM model on ETTh1, indicating that wavelet-based tokenization remains advantageous for this dataset. The next phase of the project will focus on fair comparison across multiple datasets, improved FFT tokenization, hybrid tokenization design, and long-context evaluation.

Contents

Declaration **1**

Acknowledgement **2**

Abstract **3**

1 Introduction **8**
1.1 Background . . . 8
1.2 Motivation . . . 8
1.3 Problem Statement . . . 8
1.4 Research Questions . . . 9
1.5 Objectives . . . 9

2 Literature Review **10**
2.1 Multivariate Time Series Forecasting . . . 10
2.2 SimpleTM . . . 10
2.3 Role of Tokenization . . . 10
2.4 Research Gap . . . 10

3 Proposed System **11**
3.1 Overall Idea . . . 11
3.2 Proposed Variants . . . 11
3.3 Expected Contribution . . . 11

4 System Architecture **12**
4.1 Original SimpleTM Architecture . . . 12
4.2 Proposed FFT-Only Architecture . . . 12
4.3 Planned Hybrid Architecture . . . 12
4.4 Architecture Figure Placeholder . . . 13

5 Methodology **14**
5.1 Dataset Selection . . . 14
5.2 Exploratory Data Analysis . . . 14
5.3 Tokenization Evidence . . . 14
5.4 Implementation Methodology . . . 15
5.5 Evaluation Methodology . . . 15

6 Implementation Details **16**
6.1 Repository Setup . . . 16
6.2 Files Studied . . . 16
6.3 FFT-Only Tokenizer Baseline . . . 16
6.4 Issues Found During Debugging . . . 16
6.5 Important Technical Observation . . . 17

7 Experimental Setup **18**
7.1 Initial FFT-Only Experiment . . . 18
7.2 Ablation Plan . . . 18

8 Results and Discussion **19**
8.1 FFT-Only Baseline Result . . . 19
8.2 Comparison with Original SimpleTM . . . 19
8.3 Interpretation . . . 19
8.4 Why the Current FFT Result is Still Useful . . . 19
8.5 Result Figures . . . 20

9 Work Completed So Far **22**

10 Challenges Faced **23**
10.1 Repository and Environment Issues . . . 23
10.2 Modeling Issues . . . 23

11 Future Work **24**
11.1 Short-Term Tasks . . . 24
11.2 Main Planned Extensions . . . 24
11.3 Expected Final Contribution . . . 24

12 Conclusion **25**

A Appendix A: Important Code Changes **27**
A.1 FFT-Only Tokenizer . . . 27
A.2 Loss Function Patch . . . 27
A.3 Ablation Script . . . 27

B Appendix B: Additional Figures **28**
B.1 EDA Figures . . . 28
B.2 Training Outputs . . . 28

List of Figures

4.1 Overall architecture of the proposed SimpleTM++ framework 13

8.1 Training and validation loss curves 20
8.2 Prediction versus ground truth for FFT-only model 21

List of Tables

7.1 Initial FFT-only experiment configuration 18

8.1 FFT-only baseline performance on ETTh1 19

Chapter 1

Introduction

1.1 Background

Time-series forecasting refers to the task of predicting future values of a signal using historical observations. In the multivariate setting, multiple variables evolve over time, and the objective is to predict future values by exploiting both temporal dependencies and cross-channel interactions.

Accurate multivariate forecasting is important in many applications:

- Electricity demand estimation
- Weather and climate prediction
- Traffic flow monitoring
- Industrial sensor analysis
- Financial time-series analysis

1.2 Motivation

Recent Transformer-based methods for time-series forecasting achieve strong predictive performance, but they are often large and computationally expensive. SimpleTM was proposed as a lightweight baseline that uses wavelet-inspired tokenization and geometric attention. This motivates an important question: *is one fixed tokenization scheme sufficient for all multivariate time-series datasets?*

Exploratory observations suggest that different datasets contain different kinds of structures:

- strong periodic patterns,
- local spikes and short-term fluctuations,
- non-stationary trends and multi-scale dynamics.

This motivates the investigation of alternative tokenizers such as FFT-based tokenization and, eventually, hybrid tokenization.

1.3 Problem Statement

The original SimpleTM framework uses a wavelet-based tokenization mechanism for multivariate time-series forecasting. However, real-world datasets are heterogeneous and may exhibit signal characteristics

for which alternative representations, such as frequency-domain or local convolutional representations, may be more appropriate.

The problem addressed in this project is:

> To study whether replacing or extending the original wavelet-based tokenizer in SimpleTM with FFT-based and hybrid tokenization strategies can improve representation quality and forecasting performance across diverse multivariate time-series datasets.

1.4 Research Questions

The project is guided by the following research questions:

1. Can an FFT-based tokenizer serve as a valid alternative to the original wavelet-based tokenizer in SimpleTM?
2. Are different tokenization strategies better suited to different types of datasets?
3. Can a hybrid tokenization framework provide more robust forecasting performance across datasets?
4. Does longer historical context improve the effectiveness of different tokenization strategies?

1.5 Objectives

The objectives of the project are:

1. Study the SimpleTM paper and official implementation in detail.
2. Understand the role of tokenization in multivariate time-series forecasting.
3. Perform exploratory data analysis on benchmark datasets used in SimpleTM.
4. Implement an FFT-only tokenizer baseline.
5. Compare FFT-based tokenization with the original SimpleTM model.
6. Extend the framework toward hybrid tokenization and long-context evaluation.

Chapter 2

Literature Review

2.1 Multivariate Time Series Forecasting

Multivariate time-series forecasting has traditionally been addressed using autoregressive models, state-space models, recurrent neural networks, convolutional models, and, more recently, Transformer-based approaches. Transformers have become attractive because of their ability to model long-range dependencies, but vanilla attention is often expensive and may not always preserve signal structure efficiently.

2.2 SimpleTM

SimpleTM is a lightweight model for multivariate time-series forecasting that uses signal-processing-inspired tokenization and geometric attention. The model is designed as a specialized forecasting architecture rather than a large general-purpose sequence model. The paper shows that even a one- or two-layer model can remain competitive with significantly larger forecasting architectures.

2.3 Role of Tokenization

Tokenization transforms raw time-series input into a representation that is easier for the model to process. In SimpleTM, tokenization is wavelet-based, which provides a multi-scale representation. However, signal-processing theory also suggests other possible representations:

- **Wavelet tokenization**: suitable for multi-scale and non-stationary signals.
- **FFT tokenization**: suitable for periodic and frequency-dominant signals.
- **Convolutional/local tokenization**: suitable for local motifs and sharp changes.

2.4 Research Gap

The research gap addressed by this project is that the original SimpleTM model uses a fixed tokenizer. While effective, such a design may not be equally suitable across datasets with very different underlying temporal structures. Therefore, there is a need to study:

- alternative tokenization schemes,
- fair ablations between tokenizers,
- adaptive or hybrid tokenization.

Chapter 3

Proposed System

3.1 Overall Idea

This project proposes **SimpleTM++**, an extension of the original SimpleTM framework. The idea is to preserve the lightweight forecasting structure while modifying the tokenization stage.

3.2 Proposed Variants

The project studies the following model variants:

1. **Original SimpleTM**: wavelet-based tokenization.
2. **FFT-only SimpleTM**: replaces wavelet tokenization with an FFT-based tokenizer.
3. **Conv-only SimpleTM**: uses local convolutional tokenization.
4. **Hybrid SimpleTM++**: combines wavelet, FFT, and convolutional tokenization through fusion.

3.3 Expected Contribution

The primary contribution is to analyze whether hybrid tokenization improves robustness across heterogeneous benchmark datasets compared to the original fixed-tokenizer design.

Chapter 4

System Architecture

4.1 Original SimpleTM Architecture

The original SimpleTM pipeline can be summarized as:

$$\textbf{Input Time Series} \rightarrow \textbf{Embedding} \rightarrow \textbf{Wavelet Tokenization} \rightarrow \textbf{Geometric Attention} \rightarrow \textbf{Forecast Head}$$

4.2 Proposed FFT-Only Architecture

The FFT-only variant replaces the tokenization stage:

$$\textbf{Input Time Series} \rightarrow \textbf{Embedding / Preprocessing} \rightarrow \textbf{FFT Tokenization} \rightarrow \textbf{Attention Encoder} \rightarrow \textbf{Forecast Head}$$

4.3 Planned Hybrid Architecture

The planned final architecture for SimpleTM++ is:

$$\textbf{Input Time Series} \rightarrow \left\{ \begin{array}{l} \textbf{Wavelet Branch} \\ \textbf{FFT Branch} \\ \textbf{Conv Branch} \end{array} \right. \rightarrow \textbf{Fusion / Gating} \rightarrow \textbf{Attention Encoder} \rightarrow \textbf{Forecast Head}$$

4.4 Architecture Figure Placeholder

figures/architecture.png

Figure 4.1: Overall architecture of the proposed SimpleTM++ framework

Chapter 5

Methodology

5.1 Dataset Selection

The project uses the same family of benchmark datasets commonly used for multivariate forecasting evaluation:

- ETTh1, ETTh2
- ETTm1, ETTm2
- Weather
- Electricity
- Traffic
- Solar

5.2 Exploratory Data Analysis

EDA was performed before model modification to understand dataset characteristics and justify the tokenization study. The EDA focused on:

- sample time-series plots,
- rolling mean and rolling standard deviation,
- autocorrelation analysis,
- FFT spectrum,
- wavelet decomposition,
- cross-channel correlation.

5.3 Tokenization Evidence

The EDA was interpreted using three indicators:

- **Periodicity strength**: motivates FFT-based tokenization.
- **Local volatility**: motivates convolution/local tokenization.
- **Non-stationarity proxy**: motivates wavelet/multi-scale tokenization.

5.4 Implementation Methodology

The implementation methodology followed these steps:

1. Study the original codebase and identify the tokenization layer.
2. Build a working FFT-only tokenizer baseline.
3. Modify the training pipeline to support the new tokenizer safely.
4. Run training and evaluate the model on ETTh1.
5. Compare the results against the original SimpleTM benchmark.
6. Prepare for hybrid tokenizer implementation.

5.5 Evaluation Methodology

The evaluation uses the following metrics:

- Mean Squared Error (MSE)
- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)
- Mean Absolute Percentage Error (MAPE)

  For fair comparison, the plan is to keep:

- dataset,
- context length,
- prediction horizon,
- model size,
- training schedule

fixed across ablation settings, while varying only the tokenization strategy.

Chapter 6

Implementation Details

6.1 Repository Setup

The codebase was set up in Kaggle and local development environments. The benchmark datasets were downloaded from a shared Google Drive archive and placed in the expected repository structure.

6.2 Files Studied

The key files studied and modified include:

- `run.py`
- `experiments/exp_long_term_forecasting.py`
- `data_provider/data_loader.py`
- `model/SimpleTM.py`
- `layers/SWTAttention_Family.py`

6.3 FFT-Only Tokenizer Baseline

An FFT-only tokenizer was implemented as a first alternative to the original wavelet tokenizer. The model trains successfully and serves as a valid working baseline.

6.4 Issues Found During Debugging

The main practical issues encountered were:

- dataset path mismatch after zip extraction,
- `np.Inf` incompatibility with NumPy 2,
- training failure when the loss still assumed `attn[0]` existed,
- mismatch between old shell scripts and new parser arguments,
- output inspection difficulties because checkpoints were saved as `.pth` and `.pdf`.

6.5 Important Technical Observation

The current FFT-only layer works as a baseline, but it requires refinement for a fair comparison. In particular:

- FFT should ideally be applied along the temporal axis rather than the embedding axis,
- the original attention logic should remain unchanged when only the tokenizer is being ablated,
- the configured number of heads should be respected rather than hardcoded.

Chapter 7

Experimental Setup

7.1 Initial FFT-Only Experiment

The initial working FFT-only experiment was carried out on the ETTh1 dataset.

Table 7.1: Initial FFT-only experiment configuration

Parameter	Value
Dataset	ETTh1
Input Length (`seq_len`)	96
Prediction Length (`pred_len`)	96
d_model	32
d_ff	32
Encoder Layers	1
Batch Size	32
Epochs	10
Device	CPU

7.2 Ablation Plan

The planned ablation modes are:

- `swt_only`
- `fft_only`
- `conv_only`
- `hybrid`

  Each mode will be evaluated under the same training configuration for fair comparison.

Chapter 8

Results and Discussion

8.1 FFT-Only Baseline Result

The FFT-only model trained successfully and produced the following representative result on ETTh1:

Table 8.1: FFT-only baseline performance on ETTh1

Metric	Value
MSE	0.4119
MAE	0.4228
RMSE	0.6418
MAPE	0.6995
MSPE	43769.12

8.2 Comparison with Original SimpleTM

The original SimpleTM model reports stronger performance on ETTh1, indicating that the current FFT-only baseline underperforms the original wavelet-based tokenizer on this dataset.

8.3 Interpretation

The FFT-only model is not unusable; in fact, it learns meaningful forecasting behavior and converges during training. However, the current implementation suggests that FFT-only tokenization, in its present form, is weaker than the original SimpleTM tokenizer on ETTh1.

8.4 Why the Current FFT Result is Still Useful

The FFT-only run is still valuable because:

- it proves that the modified training pipeline works,
- it provides a valid ablation baseline,
- it helps identify implementation improvements required for a fairer tokenizer comparison.

8.5 Result Figures

figures/loss_curve.png

Figure 8.1: Training and validation loss curves

figures/prediction_plot.png

Figure 8.2: Prediction versus ground truth for FFT-only model

Chapter 9

Work Completed So Far

1. Studied the SimpleTM paper and official implementation.
2. Understood the architecture and role of tokenization.
3. Built an EDA pipeline for benchmark datasets.
4. Interpreted dataset characteristics relevant to tokenizer selection.
5. Set up the codebase in Kaggle and fixed environment issues.
6. Implemented an FFT-only tokenizer baseline.
7. Resolved training-path and loss-related bugs.
8. Ran a successful FFT-only experiment on ETTh1.
9. Prepared the codebase for future ablation and hybrid extension.

Chapter 10

Challenges Faced

10.1 Repository and Environment Issues

Several issues were encountered during repository execution and adaptation:

- mismatch between extracted dataset structure and expected folder paths,
- unsupported arguments when using old scripts with new parser definitions,
- difficulty rendering checkpoint PDFs inside Kaggle,
- need for compatibility fixes for modern NumPy versions.

10.2 Modeling Issues

The FFT-only baseline highlighted modeling issues such as:

- whether FFT is applied on the correct dimension,
- loss-term assumptions inherited from the original model,
- differences between fair tokenizer replacement and full architectural modification.

Chapter 11

Future Work

The remaining work is organized into the following stages:

11.1 Short-Term Tasks

1. Run the original SimpleTM baseline under matched settings.
2. Improve the FFT-only tokenizer by applying FFT over the temporal axis.
3. Preserve original attention while changing only tokenization.
4. Use the configured number of heads correctly.

11.2 Main Planned Extensions

1. Implement a convolutional tokenizer branch.
2. Implement hybrid fusion between wavelet, FFT, and conv branches.
3. Run ablation studies across multiple datasets.
4. Study longer context lengths.

11.3 Expected Final Contribution

The final project aims to provide:

- a systematic tokenizer ablation study,
- a refined FFT-based baseline,
- a hybrid tokenization extension to SimpleTM,
- a fair empirical comparison across benchmark datasets.

Chapter 12

Conclusion

This project investigates tokenization design in multivariate time-series forecasting through the lens of the SimpleTM framework. The work completed so far establishes a strong foundation: the original model has been studied, benchmark datasets have been analyzed, the codebase has been adapted, and a working FFT-only tokenizer baseline has been implemented. Although the current FFT-only model underperforms the original SimpleTM on ETTh1, it remains an important experimental baseline and reveals key directions for improvement. The next phase of the project will focus on fair tokenizer comparison, hybrid tokenization, and broader empirical evaluation.

Bibliography

[1] Hui Chen, Viet Luong, Lopamudra Mukherjee, and Vikas Singh,
SimpleTM: A Simple Baseline for Multivariate Time Series Forecasting,
International Conference on Learning Representations (ICLR), 2025.
Available at: https://proceedings.iclr.cc/paper_files/paper/2025/hash/27c546ab1e4f1d7d638e6a8dfbad9a07-Paper-Conference.pdf

[2] Singh Group,
SimpleTM Official Implementation,
GitHub repository.
Available at: https://github.com/vsingh-group/SimpleTM

[3] Ashish Vaswani et al.,
Attention Is All You Need,
NeurIPS, 2017.

[4] A. V. Oppenheim and R. W. Schafer,
Discrete-Time Signal Processing,
Prentice Hall.

Appendix A

Appendix A: Important Code Changes

A.1 FFT-Only Tokenizer

Paste your final FFT-only tokenizer code here.

A.2 Loss Function Patch

Paste the modified loss block used to avoid failure when `attn` is `None`.

A.3 Ablation Script

Paste your shell script for:

- `hybrid`
- `swt_only`
- `fft_only`
- `conv_only`

Appendix B

Appendix B: Additional Figures

B.1 EDA Figures

Add FFT spectrum, rolling statistics, and correlation plots here.

B.2 Training Outputs

Add screenshots of logs, checkpoints, and prediction outputs here.